

Guia de Deploy

Site institucional — DMS Desempenho Socioambiental

Passo a passo para publicar o site em produção e garantir que tudo funcione (domínio, hospedagem e formulário de contato). Atualizado em 29/06/2026.

1. Visão geral

O site é uma aplicação **Node.js própria**, sem framework (não usa Express) e sem nenhuma dependência externa instalada via npm. Não existe banco de dados: o carrossel, as logos de clientes/parceiros e os vídeos de experiências são lidos diretamente da pasta `assets/` pelo próprio servidor a cada acesso.

Por isso, hospedagem **puramente estática** (GitHub Pages, Website Builder da GoDaddy, etc.) **não funciona** — é necessário um serviço que mantenha um processo Node.js rodando o tempo todo. O projeto exige Node.js 20 ou superior.

Você já tem o domínio registrado na GoDaddy mas ainda não escolheu onde hospedar o site. Este guia recomenda o **Render** como caminho principal — é gratuito para começar, tem deploy automático via GitHub e emite certificado SSL sozinho, o que o torna o caminho mais simples para quem não administra servidor. No final há uma alternativa para quem preferir manter tudo dentro da própria GoDaddy.

2. O que você vai precisar antes de começar

- Uma conta no **GitHub** (gratuita) — é para onde o código do site vai ser enviado.
- Uma conta no **Render** (gratuita) — em render.com.
- As **credenciais SMTP** do e-mail que envia as mensagens do formulário de contato (servidor, porta, usuário e senha) — são as mesmas já configuradas no arquivo `.env` do projeto, na sua máquina.
- Acesso ao painel da **GoDaddy** (login da conta onde o domínio está registrado).

3. Passo 1 — Preparar o projeto

3.1 Nunca envie o arquivo `.env`

O projeto já tem um arquivo `.gitignore` que exclui o `.env` e a pasta `node_modules` de qualquer envio. Confirme que isso continua assim — esse arquivo guarda a senha do e-mail SMTP e não pode ir para um repositório público.

3.2 Recomendado: não envie pastas de uso interno

As pastas e arquivos abaixo foram usados durante o desenvolvimento e não fazem parte do site público. O próprio servidor já bloqueia o acesso externo a eles mesmo que continuem na pasta, mas é mais limpo deixá-los fora do repositório que vai para a hospedagem:

```
tools/  
chrome-* -profile/  
edge-manual-profile/  
ux-audit/  
conversion-audit/  
multi-audit/  
ppt_extract_*  
demo.html, demo-styles.css  
*.log, *.bak*, _qa-*.png
```

3.3 Suba o projeto para um repositório no GitHub

Crie um repositório novo (pode ser privado) em `github.com/new`. Depois, dentro da pasta do projeto, rode:

```
git init  
git add .  
git commit -m "Site DMS"  
git branch -M main  
git remote add origin <URL do repositório que o GitHub mostrar>  
git push -u origin main
```

Atenção: se o GitHub pedir login, use sua conta pessoal. Não é preciso conhecimento avançado de Git — esses seis comandos, na ordem, são suficientes.

4. Passo 2 — Criar o serviço no Render

- 1 Acesse **render.com** e faça login com sua conta do GitHub.
- 2 Clique em **New +** → **Web Service** e selecione o repositório do site.
- 3 Configure o serviço com estes valores exatos:

Runtime	Node
Build Command	npm install
Start Command	node server.js
Plano	Free (para testar) ou Starter — recomendado para o site não “dormir” entre acessos

Clique em **Create Web Service** e aguarde o primeiro deploy terminar (alguns minutos).

Por quê o plano Starter: no plano Free, o serviço “adormece” depois de um período sem acessos e demora alguns segundos para responder ao primeiro visitante seguinte. Para um site institucional em produção, o plano pago evita esse atraso — mas o Free funciona para testar tudo primeiro.

5. Passo 3 — Configurar as variáveis de ambiente

No painel do serviço, dentro do Render, vá em **Environment** → **Add Environment Variable** e cadastre as mesmas chaves que já existem no seu arquivo `.env` local:

```
PORT (opcional – o Render já define isso automaticamente)
SMTP_HOST
SMTP_PORT
SMTP_SECURE
SMTP_STARTTLS
SMTP_USER
SMTP_PASS
CONTACT_TO
CONTACT_FROM
SMTP_DOMAIN
```

Importante: copie os valores reais direto do seu arquivo `.env` local. Nunca digite a senha de SMTP em chat, planilha ou qualquer outro documento — cole apenas no campo de variável de ambiente do Render.

Obrigatórias: se faltar SMTP_HOST, SMTP_PORT, SMTP_USER ou SMTP_PASS, o formulário de contato do site passa a responder com erro 503 — essas quatro são obrigatórias para o e-mail funcionar.

Depois de salvar, o Render reinicia o serviço automaticamente para aplicar as novas variáveis.

6. Passo 4 — Apontar o domínio da GoDaddy para o Render

- 1 No Render: **Settings** → **Custom Domains** → **Add Custom Domain**. Digite o domínio raiz (ex.: dmsocioambiental.com) e também a versão com www.
- 2 O Render mostra os registros de DNS exatos que você precisa criar — normalmente um **CNAME** para o www e um registro **A** ou **ALIAS/ANAME** para o domínio raiz.
- 3 No painel da GoDaddy: **Meus Produtos** → **DNS** → **Gerenciar Zona DNS** → adicione ou edite os registros exatamente como o Render indicou.
- 4 Aguarde a propagação do DNS (de alguns minutos a até 48 horas). O Render emite o certificado SSL automaticamente assim que o domínio responder — não é preciso comprar nem instalar certificado à parte.

7. Passo 5 — Testar tudo em produção

- Abra o site pelo domínio final usando `https://` e confirme o cadeado de conexão segura no navegador.
- Teste o formulário de contato de ponta a ponta e confirme que o e-mail chega no endereço configurado em `CONTACT_TO`.
- Confirme que carrossel, logos de clientes/parceiros e vídeos de experiências aparecem normalmente (dependem da pasta `assets/` ter sido enviada completa ao repositório).
- Abra o site também em um celular para confirmar o layout responsivo.

8. Checklist final

	Repositório no GitHub criado, sem o arquivo .env
	Web Service criado no Render com Build/Start Command corretos
	Todas as variáveis SMTP cadastradas no Render
	Domínio apontado na GoDaddy e certificado SSL ativo (cadeado no navegador)
	Formulário de contato testado e e-mail recebido
	Carrossel, logos e experiências carregando normalmente
	Site testado em um celular

9. Problemas comuns

Sintoma	Causa provável
Erro 503 no formulário de contato	Falta alguma variável SMTP obrigatória no Render (Passo 3).
Site não abre / erro 502	Confira se o Start Command é exatamente "node server.js" e veja os logs do serviço no Render.
Domínio não carrega	DNS ainda propagando. Use um verificador de propagação (ex.: whatsmydns.net) ou aguarde mais tempo.
E-mail do formulário não chega	Confira a caixa de spam e usuário/senha SMTP. Provedores como Gmail e Outlook costumam exigir uma "senha de aplicativo" em vez da senha normal da conta.
Logos/carrossel não aparecem	Confirme que a pasta assets/ subiu completa para o repositório (ela é grande, ~50 MB).

10. Segurança — pontos que não pode pular

- Nunca suba o arquivo .env para o GitHub nem cole a senha SMTP em chats, planilhas ou documentos compartilhados.
- Se a senha SMTP já foi exposta em algum lugar (e-mail, mensagem, documento), troque-a antes de colocar o site no ar.
- O próprio código do site já bloqueia o acesso público a arquivos sensíveis (.env, server.js, package.json, arquivos de log) — não é necessário configurar nada extra de segurança nesse ponto, apenas não remover nem contornar essa proteção ao configurar a hospedagem.

Apêndice — Alternativa: hospedar direto na GoDaddy

Se preferir manter tudo dentro de um único provedor em vez de usar o Render:

- 1 Confirme que seu plano GoDaddy tem **cPanel** com a ferramenta “**Setup Node.js App**” (disponível em planos de hospedagem Linux/Negócios; planos básicos de Website Builder não servem para isso).
- 2 No cPanel: **Setup Node.js App** → **Create Application** → escolha a versão Node 20 ou superior, aponte para a pasta do projeto e defina `server.js` como arquivo de inicialização.
- 3 Cadastre as mesmas variáveis de ambiente do Passo 3 direto na própria tela da ferramenta.
- 4 Envie os arquivos do projeto via File Manager ou FTP — exceto o `.env`, que deve ser recriado manualmente dentro da pasta, com permissão restrita.
- 5 Ative o certificado SSL gratuito da GoDaddy (AutoSSL) em **SSL/TLS Status** para o domínio.

Diferença importante: esse caminho não tem deploy automático via GitHub — cada atualização do site exige reenviar os arquivos manualmente. Por isso o Render é o caminho recomendado no corpo deste guia.

DMS Desempenho Socioambiental — guia de deploy gerado para uso interno.